Abstract

In the high-frequency retail sector, growth is traditionally stalled by households purchasing the same subset of products repeatedly. While standard recommendation engines reinforce these existing patterns, true commercial value lies in basket expansion through encouraging product discovery and tapping into latent demand (Catalina, 2026).

In this project, we formulate product recommendation as a binary classification task using implicit feedback signals. Rather than optimising for exact ranks, the models recommend a basket of the Top-K products with the highest predicted purchase probabilities. Performance is evaluated based on whether the recommended items correspond to actual future purchases. Inclusion within the basket is prioritised over the order in which items appear to align with the primary goal of item inclusion within a grocery trip.

Using the Dunnhumby Complete Journey dataset, we integrate longitudinal transaction logs with household demographics and product attributes into an end-to-end recommendation pipeline, which is used to benchmark traditional models like Matrix Factorisation (MF) and Neural Collaborative Filtering (NCF) against User and Item Feature models (He et al., 2017), as well as standard and Multi-Branch Two Tower (MB-TT) architectures.

The project ultimately aims to identify the most effective recommendation framework for varying customer segments while demonstrating how machine learning can create measurable commercial impact through more diverse basket recommendations.

Some noteworthy highlights include:

- Personalisation: Incorporating household and demographic features significantly improves performance. The User-Feature model achieved the best performance, with an AUC of 0.9305 and recall@10 of 0.7299.
- Feature Significance: Two-Tower ablation study revealed that item-side features contribute 70.7% of the model's ranking ability, underscoring the importance of content-based attributes
- Cold-Start Robustness: To mitigate warm-start bias and user-ID memorisation, a cold-start evaluation set was created and used in an MB-TT architecture that improved Recall@10 from 0.6242 to 0.6251. Generic recommendations were reduced, enhancing diversity for new shoppers.

1. Feature Engineering

1.1 Dataset Overview

Our dataset comprises three relational tables:

Table	Description	Key Columns	Records
transaction_data.csv	Purchase transactions	household_key, product_id, sales_value, quantity, retail_disc, coupon_disc, coupon_match_disc, week_no	2,595,732
product.csv	Product catalog	product_id, manufacturer, department, brand, commodity_desc, sub_commodity_desc	(product catalog)
hh_demographic.csv	Household demographics	household_key, age_desc, income_desc, marital_status, homeowner_desc, hh_comp_desc	801

Table 1: *Overview of the Dunnhumby Complete Journey dataset structure*

The transaction data spans 711 days (approximately 102 weeks) and captures detailed purchase information, including sales values, quantities, retail discounts, manufacturer coupon discounts, and store coupon match discounts. The product table provides a hierarchical categorisation with 303 commodity descriptions and 2,260 sub-commodity descriptions across multiple departments. Brand information distinguishes between National brands (available at multiple retailers) and Private labels (store-exclusive brands).

1.2 Data Merging and Cleaning

1.2.1 Data Merging and Integration

The three datasets were merged using left joins on shared keys (product_id and household_key) to create a unified master dataset. The merge preserved all 2,595,732 transaction records. A critical observation during integration was that 45.01% of transactions, amounting to 1,168,429 data entries, lacked demographic information, indicating that not all households in the transaction data had corresponding entries in the demographic table. This demographic information includes age description, marital status, income and household size description. With many of the missing values being represented with different values like ‘Unknown’ or ‘None’, all were initially converted to NA values for easier data cleaning and identification.

On top of missing demographic data, another crucial area where missing data was found was for the product descriptions. A total of 7,839 missing entries were found for Department, Commodity Description and Sub-Commodity Description. Additionally, 2,840 data entries also had zero quantity. 16 data entries were also found to have a retail discount of more than 0. Since it is not likely that the customer was paid for purchasing the item, these 16 entries were treated as accidental wrong data entries.

1.2.1 Data Cleaning

With more than 1 million data entries being NA values for the demographic features, we wanted to determine whether they were all from the same data entries or were spread out, resulting in a serious problem of insufficient raw data available for training. However, the missing value analysis revealed structured patterns. The demographic variables that had NA values exhibited perfect correlation in their missingness patterns, confirming that missing demographics were probably due to households opting out of providing their details. Data entries that had NA values for Department also had missing values for Commodity and Sub-Commodity, where all 7,839 missing values for the three variables came from the same data entries.

While most of the NA values came from the same set of data entries, they are still a significant set of data. To handle this, two main options can be considered: prediction or removal. Should the missing values be predicted, more than half of the demographic distributions would be artificially created. This might blur the unique characteristics of specific customer segments, making the model less sensitive to nuances. As cold-start models rely heavily on user attributes, should the data points be predicted, algorithm bias from the regression algorithm may potentially be inherited by the final recommendation model, potentially masking the niche, long-tail consumption habits that define unique users' profiles. By training only on ground truth data, authentic relationships between users and items can be learnt better, leading to a higher precision when recommending products and allowing the model to handle cold-start scenarios better.

Additionally, by removing data entries with zero values, division by zero errors can be avoided. It will also prevent the data entries from introducing noise into the model, avoiding the risk of overfitting on erroneous data.

Eventually, after a strict removal of incomplete data, our dataset still remains robust with 1,283,376 interactions. A total of 728 unique households were preserved out of 801 unique

households. This provides sufficient density for high-capacity models like NCF and Two-Tower architectures to converge without needing synthetic data. Hallucinated preferences can also be avoided where imputation would force the model to learn from predicted behaviours rather than actual transactions. By filtering, we ensure that every weight update in the neural network is based on genuine customer actions.

1.3 Exploratory Data Analysis

1.3.1 Transaction Value Distribution

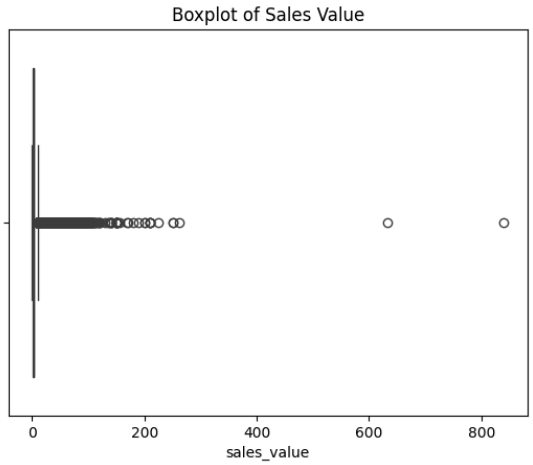

Figure 1: *Boxplot of Transaction Sales Values*

The sales value, which represents the amount received by the retailer, is extremely right-skewed. With a mean of $3.18 and a range from $0 to $840. This aligns with typical retail behaviour, where most transactions consist of low-value items, supplemented by occasional high-value 'outliers' in a customer's basket.

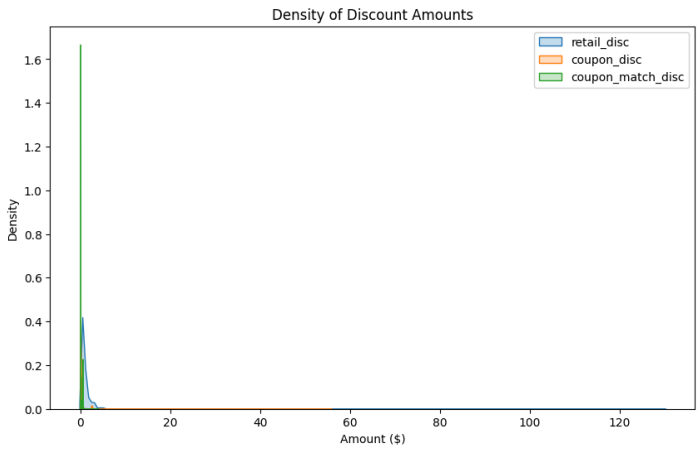

Figure 2: *Probability Density Distribution of Discount Amounts*

As coupons were offered, not all customers paid the full amount for the products. There were a total of three different discounts offered. Retail discount for members, coupon discount by manufacturers, and coupon match discounts by the retailer. The distribution of such discounts follows closely the actual value of the products, ranging from $0 to $130. All of the values stored are also negative in value.

While sales value represents the final amount that was received by the retailer, it does not reflect the actual shelf price of the products, as well as the final amount paid by the customers. This is because sales value does not include discounts offered by coupons given by manufacturers, represented by coupon_disc. The final calculation for the amount customers paid is given by the following equations, separated according to whether they are members or non-members.

For all calculations, discount variables were converted to absolute values so that they represented the magnitude of discount applied, and all prices were expressed on a per-unit basis by dividing transaction-level values by quantity.

For member transactions, the reconstructed shelf price is reconstructed as follows:

$$Member\ Shelf\ Price\ = \frac{sales_value + coupon_disc}{quantity}$$

and the actual amount paid by the customer is:

$$Member\ Paid\ Price\ = Member\ Shelf\ Price\ - \frac{coupon_disc + coupon_match_disc + retail_disc}{quantity}$$

For non-member transactions, the reconstructed shelf price is calculated as:

$$Non - member\ Shelf\ Price\ = \frac{sales_value + retail_disc + coupon_match_disc}{quantity}$$

and the actual amount paid by the customer is:

$$Non - member\ Paid\ Price\ = Non - member\ Shelf\ Price\ - \frac{coupon_disc + coupon_match_disc}{quantity}$$

The final variable, actual_customer_paid, was then assigned using the member formula when retail_disc > 0, and the non-member formula otherwise. These reconstructed price variables allow us to distinguish between the retailer-facing transaction value, the original shelf price, and the customer-facing paid price, which is important for downstream price-related features such as average spend and price sensitivity.

From the calculation of shelf price, 12,916 data entries were found to have a value of $0. Most of them were from the category named “Coupon / Misc Items”. While they were genuine transactions, a shelf price of $0 will likely represent promotional samples or coupons given out for free to customers. These free transactions will introduce significant noise where it is not possible to discern whether a consumer selected the product due to preference or because it was offered at no cost. Additionally, a zero-dollar denominator will also result in division by zero errors. By removing these entries, the model will be able to focus strictly on paid purchase intent where price and preferences are actively weighed by the consumer. After removing the transactions, the total data entries count is 1,283,376, still sufficiently dense to maintain the model's predictive power.

1.3.2 Product Category

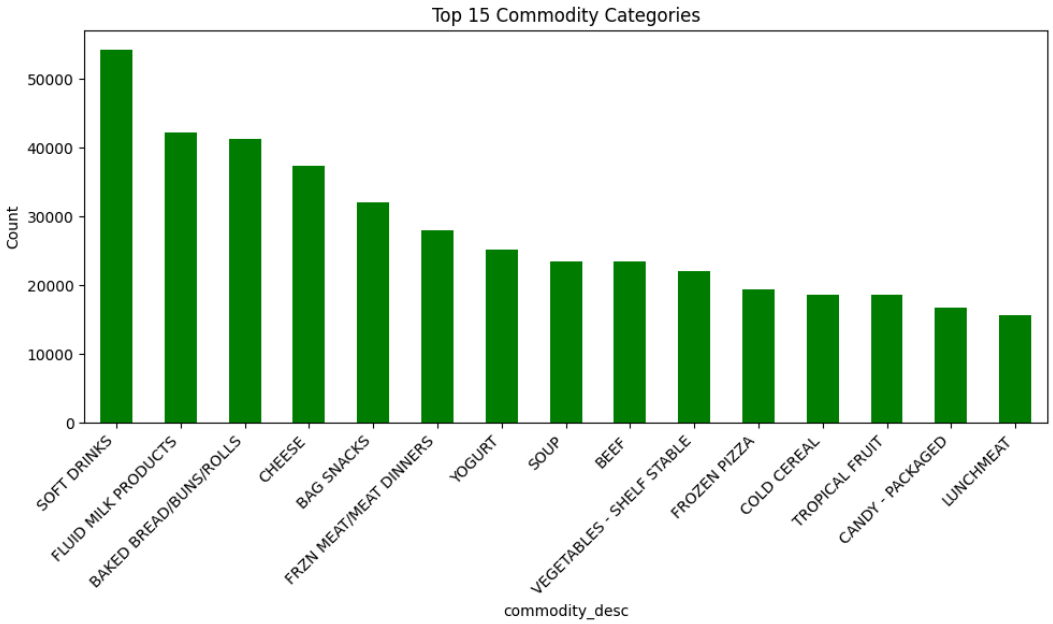

Figure 3: *Frequency Distribution of Top 15 Commodity Categories*

The product exhibits high cardinality, spanning 303 commodities and 2,260 sub-commodities. While this depth provides a rich feature set, it introduces significant data sparsity risks. High-volume categories like 'Soft Drinks', at 54,331 transactions, risk dwarfing niche categories like 'Bouquet (Non Rose)' at 3 transactions.

In traditional Collaborative Filtering, these low-frequency items often suffer from poorly defined embeddings due to a lack of interaction pairs. To mitigate this, a Two-Tower architecture will need to be employed. By transitioning from ID-based memorisation to attribute-based learning, the model can leverage the shared features of sub-commodities.

This allows the system to borrow signals from high-volume neighbours to make accurate predictions for the long tail, ensuring robust performance even for the least frequent items.

1.3.3 Brand Analysis

The products come from 5,411 unique manufacturers. These manufacturers are categorised into Private and National. Private refers to brands that are only found in the retailer, while National brands can be found in other retailers too. This will likely refer to big-name brands and house brands.

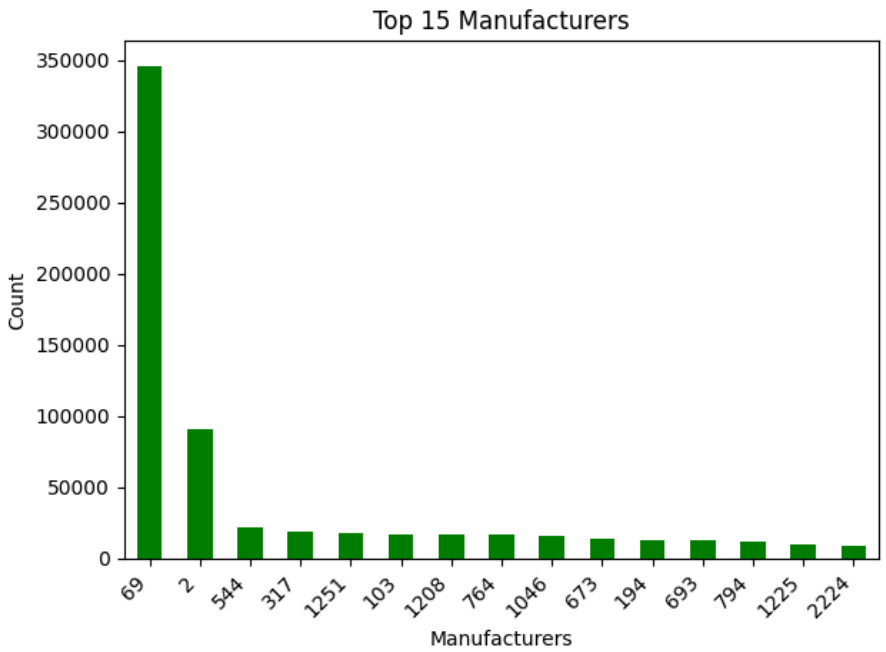

Figure 4: *Frequency Distribution of the Top 15 Manufacturers*

The dataset distribution mirrors a typical retail outlet. National brands account for 71.8% of volume, at 930,778 transactions, while Private labels represent the remaining 28.2%, at 365,514 transactions. These Private brands function as a strategic lever for value-seeking segments. Products are offered at a lower price point, targeting price-sensitive or brand-agnostic households. For our recommendation model, distinguishing between these two brand types is critical for encoding price elasticity and household-specific value priorities.

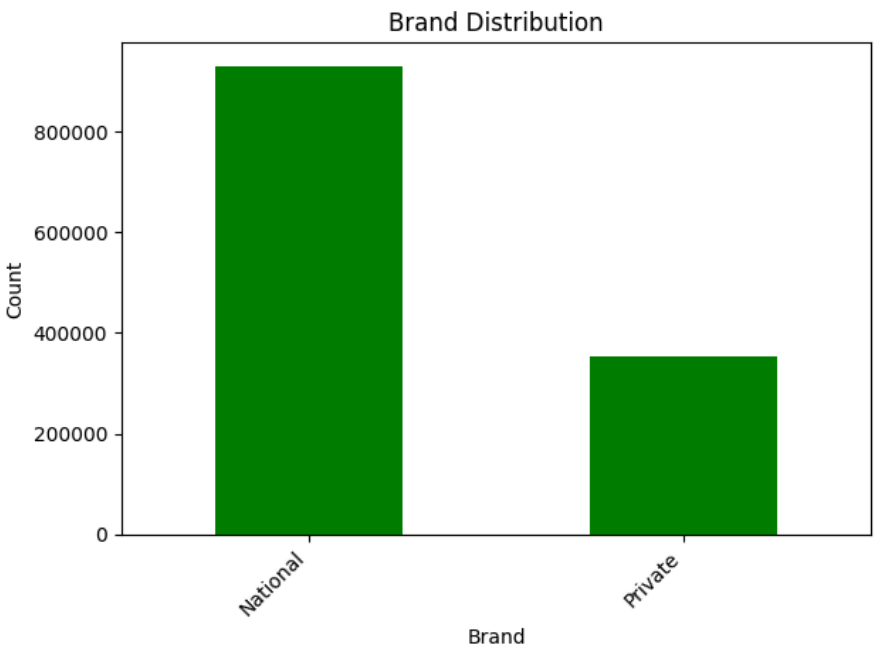

Figure 5: *Distribution of Transactions by Brand Type*

The distribution of products among manufacturers is also relatively consistent, with only one outlier accounting for 358,571 items. This concentration likely represents the Private label producer, aligning with the brand type analysis. Outside of the outlier, the balanced distribution prevents the model from developing a brand bias. Cross-manufacturer attributes can be picked up by the model, allowing a wider variety of products to be suggested, increasing the chances of customers converting a basket recommendation into a transaction.

1.3.4 Household Demographics

The final cleaned dataset contains 728 unique households of different family structures, from households with kids to single-person households.

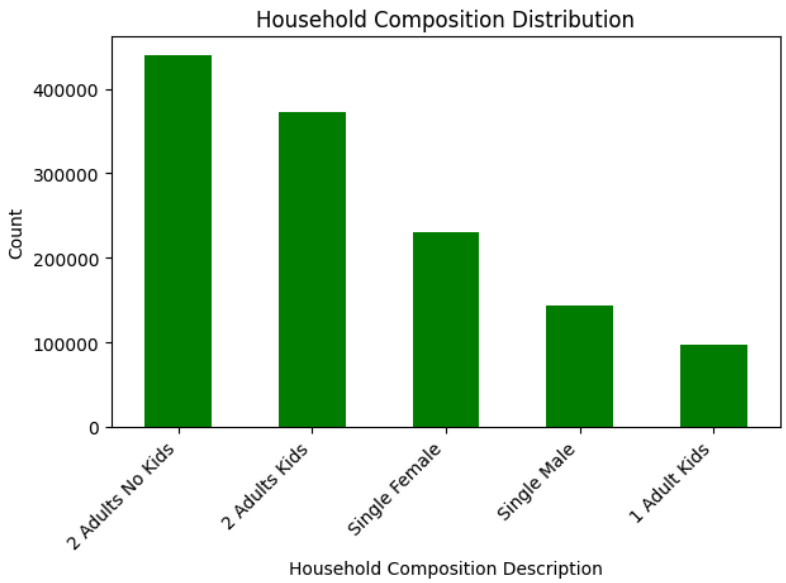

Figure 6: *Frequency Distribution of Household Composition*

The overall household is dominated by 2 Adults, No Kids, which takes up 34.3% of total transactions at 444,074 transactions. The smallest group is Single Adults with No Kids at only 7.6%, or 98,649 transactions. While the distribution is mainly centred around families, either with kids or no kids, this demographic data is still sufficient to group users into similar categories where the smallest group will not be treated as an outlier. This will be especially helpful for cold users with minimum transactions, allowing recommendations to be more targeted and inclusive to all customer groups.

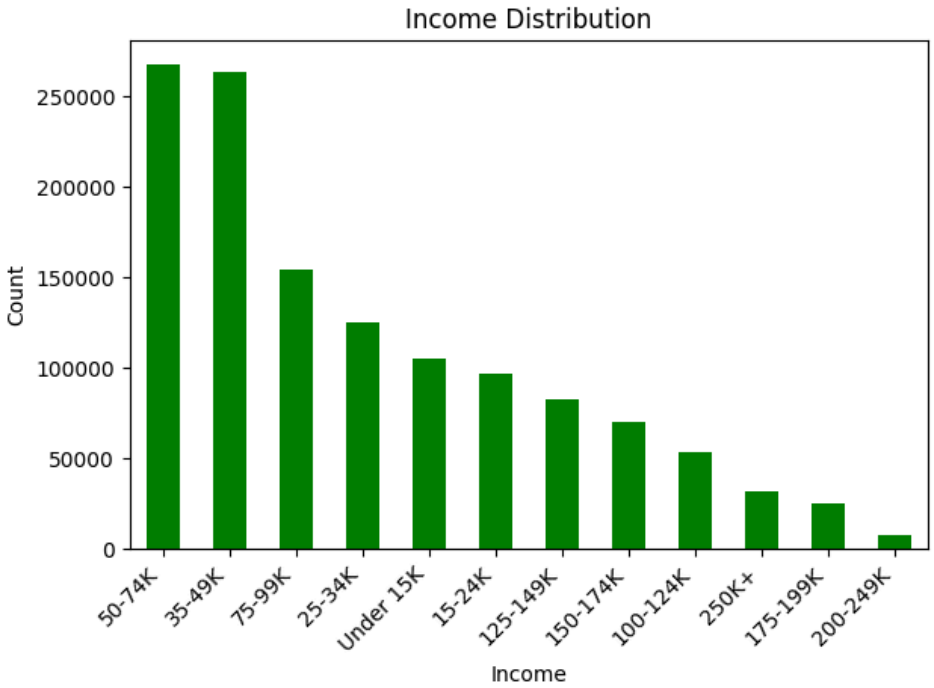

Figure 7: *Frequency Distribution of Income*

While preferences or wants are important, affordability is another aspect that has to be considered. With a fair distribution across different income ranges, it will allow the model to move beyond what customers like to what they can afford. With over 536,000 transactions between the 35k and 74k brackets, it shows that there is a great middle-market dominance. This provides the model with a robust baseline for understanding value-driven consumption and price-sensitive shopping patterns. The fair representation of higher-income households will also allow the model to learn to associate the income level with a lower friction toward higher-priced items. This may allow the model to move away from recommending popular, affordable items and lean towards more ambitious cross-category recommendations like premium organic products or a preference for National brands instead. Ultimately, by integrating income as a feature, the model can learn when to prioritise quality and brand prestige and when to focus on value and Private label alternatives.

1.4 Feature Engineering

For the model to be able to effectively recommend products accurately for both warm and cold scenarios, our feature engineering strategy focuses on transforming raw transactional data into high-dimensional representations of household preferences and product characteristics. The goal was to use specific domain knowledge to capture behavioural patterns, demographic characteristics, and product attributes that go beyond raw transaction data.

1.4.1 Demographic Feature - Life Stage

To capture the impact of household evolution on consumer behaviour, we have formulated a life_stage variable consisting of nine discrete categories. This indicator is synthesised through the interaction of three primary features:

1. age_desc
2. hh_comp
3. kid_category_desc

which form age, number of adults, and number of kids, respectively.

The different life stages are as follows:

- 0: Young family
- 1: Older family
- 2: Others with kids
- 3: Young single
- 4: Middle-aged single

- 5: Retired/Older single
- 6: Retired/Older couple
- 7: Couple without kids
- 8: Others

While there are significant groups that might dominate each set, like category 7, which represents couples without kids, there is a fair representation of most categories throughout for all three sets. When categories 0, 1, and 2 are all added up to represent families, they will take up the biggest proportion in all of the sets. This reflects the standard retail volume bias wherein, in typical supermarkets, families account for most of the transactions in terms of quantity and total amount spent. By having the recommendation model prioritise this group, possible business benefits can be brought where the transaction value obtained can be increased.

1.4.2 Recency and Decay

To account for the changes in consumer preferences, we implemented a temporal decay function to weight historical transactions. This was derived from the raw data, week_no, or week number. Unlike standard frequency counts, which treat all interactions as equal, recency weighting applies a diminishing coefficient to older data points, thereby prioritising a household's most recent purchases. This allows the model to internalise the longitudinal patterns within the time series, enabling it to distinguish between long-term core habits and temporary preferences.

1.4.3 Price Sensitivity

In order to capture the customer’s responsiveness to changes in price, we engineered the feature price_sensitivity_index. It is a composite score (0 - 9) based on discount usage that ranks users based on how much they respond to discounts relative to the rest of the households. Low values represent households with price-inelastic demand, and high values represent households with price-elastic demand who hunt for discounts. In order to construct the feature, the prices of products were categorised into different quantiles. The average discount enjoyed by a customer was also calculated for the train set. Finally, the membership status was also referred to since it provides an additional discount on top of the vouchers provided by the retailer or manufacturers.

By introducing this feature, it helps to encapsulate all of the price-related signals that might influence customers' shopping behaviours. It eventually reduces the number of features that have to be incorporated, reducing dimensionality and multicollinearity, allowing the

recommendation model to more efficiently align product suggestions with a user's underlying economic behaviour. Discounted inventory can also be surfaced to the most receptive customer segments, boosting the effects of sales campaigns.

1.4.4 Average Spent

The feature avg_spent_per_trip is the mean transaction value per shopping trip. This feature works as a proxy, representing avg_basket_size and spending_capacity, allowing the model to differentiate between high-value bulk shoppers and low-frequency consumers.

1.4.5 Frequency of Purchases

To capture temporal purchasing behavior, we engineered a set of features based on the time intervals between transactions. These include measures such as day_since_last_item, days_since_any_purchase, avg_item_repurchase_gap, user_avg_gap, and item_avg_gap. Together, these features aim to model individual shopping rhythms and product-specific repurchase cycles, which are particularly relevant in grocery settings where many items are bought on a recurring basis.

Building on these temporal signals, we introduced additional indicators such as item_first_purchase and user_first_purchase, which explicitly flag cold-start scenarios. These were further combined into a composite feature, cold_start_severity, to quantify the extent of cold-start conditions for each interaction (ranging from fully warm to completely cold). This allowed us to analyze how frequently the dataset contains first-time purchases and to assess the severity of cold-start challenges.

Empirically, we observed that a substantial proportion of transactions correspond to first-time item purchases. This suggests that the dataset naturally contains strong cold-start characteristics, where historical interaction signals are limited. In such cases, the model must rely more heavily on alternative sources of information, such as user demographics and product attributes, to make meaningful predictions.

However, despite their interpretability, these temporal and cold-start features were ultimately not included in the final model. This is because their contribution to predictive performance was limited relative to other feature groups, particularly high-dimensional product embeddings and engineered user features. Additionally, some of these features introduced sparsity and noise, especially for infrequent users or items, which reduced their effectiveness in improving model performance.

Nevertheless, this analysis provides useful insight into the structure of the dataset and highlights the importance of feature-based generalisation in handling cold-start scenarios, even when explicit temporal signals are not directly utilised in the final model.

1.4.6 Brand Loyalty

To better distinguish household preferences, the feature user_private_label_loyalty was created from is_private_label, which is binary in value. This feature helps to capture the proportion of private-label purchases per household, allowing the model to differentiate between price-sensitive value shoppers and brand-conscious consumers. This feature will also help the User Tower in the Two Tower model to effectively tailor recommendations toward the appropriate brand tier, aligning them with the household's economic behaviour, hopefully improving Recall@10 values.

1.4.7 Co-occurrence

To capture the complementary nature of goods, the feature log_co_purchase is created by first aggregating transactions into daily household baskets and calculating a raw co_purchase_count, which represents the total number of other items in the same basket with a specific product across the entire training set. The higher the value, the greater the complementary good it is, and hence whether the good is usually purchased with another item.

An average trip of a user might consist of different amounts of goods being purchased. During some trips, users are only there to buy a few items, usually in and out quickly. However, in other trips, users are stocking up and purchasing multiple items. With there being different types of grocery trips that a user will do, the items purchased during these trips will therefore also be different. This can be highlighted through the avg_basket_size. Items can be determined to be usually purchased during large restock trips or quick runs, making recommendations better and adjusted according to the current basket size.

Similarly, the feature basket_centrality looks at determining how popular an item is, where it is calculated based on the number of users who have it in their baskets. This helps with cold start recommendations, where if the model does not know too much about the user, the most common item that is purchased might be recommended instead.

1.4.8 Commodity Entropy

Commodity entropy is captured by the feature shannon_entropy. It is used to quantify the variety and predictability of each household's purchasing behaviour. By calculating the entropy of commodity descriptions at the household level, we capture a shopping diversity score that distinguishes habitual consumers from broad-spectrum explorers. This feature provides a critical behavioral signal where low entropy users receive high precision recommendations based on recurring needs, while high entropy users are served a more diverse set of items. For cold start users, the application of global_entropy ensures a stable behavioral baseline, allowing the model to generalise based on aggregate population trends until individual purchase signatures appear.

1.4.9 Encoding Categorical Features

To enable the processing of non-numeric data within the recommendation architecture, label encoding is applied to all primary categorical features: department, brand, age bracket, income bracket, and household size. Encoders are fitted exclusively on the training distribution, ensuring a consistent category-to-index mapping is preserved across all temporal splits and preventing data leakage from future observations.

For models that leverage embedding layers, these integer indices serve as lookup keys into dense vector spaces learned during training, enabling the model to capture latent relationships between categories. For simpler models, the integer-encoded representation provides a lightweight yet consistent numerical input without additional parameterisation.

1.4.10 Open AI embeddings

To capture the product’s relationship based on their descriptions, we used OpenAI sentence embeddings generated from a concatenation of the item’s category and sub-category names. These embeddings capture semantic similarity between products, allowing the model to generalize to items with similar descriptions even if they have limited interaction history. Since the original embedding dimension is high, Principal Component Analysis (PCA) was applied to reduce dimensionality to 256 components. This reduces computational cost while retaining most of the semantic information.

1.4.11 Scaling Continuous Features

Continuous features such as entropy, spending, and frequency were scaled using RobustScaler.

RobustScaler centres each feature around the median and scales it using the interquartile range. This is preferred over standard scaling because transaction data often contains extreme values, such as unusually large purchases, which can distort mean-based scaling. This transformation improves numerical stability and helps neural networks converge more effectively.

1.4.12 Encoding User and Item IDs

User IDs and item IDs were encoded as integer indices and used as inputs to embedding layers in models such as Matrix Factorisation and Neural Collaborative Filtering.
These embeddings allow the model to learn latent representations of users and items based on interaction patterns. Each user and item is mapped to a dense vector that is optimised during training.

1.4.13 Feature Selection

Following feature engineering, we performed feature selection to retain only the most relevant, non-redundant signals for model training. Our objective was to balance informativeness, interpretability, and computational efficiency while avoiding multicollinearity and noise.

We prioritised features that capture distinct behavioural, demographic, and product-level information. Correlation analysis showed low pairwise correlations, indicating minimal redundancy and that each feature contributes complementary information to the model.

Feature selection was further guided by domain relevance:

- User behavioral and demographic features such as income_desc, decay_freq, commodity_entropy, regularity, and avg_spend_per_trip capture purchasing power, recency effects, diversity of preferences, shopping consistency, and basket size
- Derived demographic features, such as life_stage, consolidated age, household size, and family composition, into a single interpretable variable, capturing lifecycle-driven consumption patterns
- Price-related behavior, captured by price_sensitivity_index and user_private_label_loyalty, reflects responsiveness to promotions and brand preferences
- Item-level features, including log_co_purchase, avg_basket_size, and basket_centrality, capture product relationships, typical purchase context, and item popularity

- Price and temporal item features, such as item_price_tier and item_avg_gap, provide signals on affordability and replenishment cycles, while item_first_purchase supports cold-start analysis

Features that were sparse, redundant, or unstable were excluded to improve generalisation:

- Transaction-level variables (sales_value, quantity) were omitted as they do not represent stable user or item characteristics; instead, aggregated features (e.g. avg_spend_per_trip) were used
- Raw discount features (coupon_disc, coupon_match_disc, retail_disc) were excluded due to noise, and replaced with the more robust price_sensitivity_index
- The raw temporal index (week_no) was removed in favour of engineered temporal features such as decay_freq and regularity

Overall, this selection process ensures that the final feature set captures rich, complementary signals while remaining compact and efficient, enabling better model generalisation and stability.

1.5 Negative Sampling Strategy

Since our problem is formulated as a binary classification task where only positive interactions (purchases) are observed, negative sampling was introduced to generate instances of non-interactions.

For each observed user-item interaction, a set of items that the user has not interacted with was randomly sampled as negative examples. This helps the model learn to distinguish between items that users are likely to purchase and those they are not.

A negative sampling ratio of four was chosen to balance training efficiency and model performance, a heuristic widely supported in implicit feedback literature (He et al., 2017). Using too few negative samples limits the model's ability to distinguish between relevant and irrelevant items. However, using too many negatives significantly increases computational cost without proportional gains in performance. A ratio of four provides sufficient contrast between positive and negative examples while maintaining efficient training and is commonly used in implicit feedback recommendation settings.

In standard negative sampling, negatives are drawn uniformly from items in the training set. This results in very few cold items appearing as negatives. For the Multi-Branch Two-Tower, this creates an issue because the cold item branch receives an insufficient training signal.

To address this, 15% of negative samples are explicitly drawn from cold items. This ensures that the cold branch learns to correctly assign low scores to irrelevant cold items and does not overfit to sparse positive signals.

1.6 Train-Validation-Test Split

A temporal split was employed instead of a random shuffle so that training, validation, and test sets follow the true chronological order of user-item interactions. By partitioning the data chronologically into training, validation, and test sets, we strictly prevent look-ahead bias, where future transaction patterns inadvertently leak into the training phase. This better reflects the real-world recommendation setting, where models must predict future purchases from past behaviour.

To ensure that performance differences are not driven by distributional shifts, we verified that all three splits remain broadly comparable across several key dimensions. First, user overlap analysis shows that all users in the validation and test sets are warm users (all users in these sets have prior interaction history in the training set). This ensures that evaluation reflects a consistent user base rather than introducing unseen-user effects.

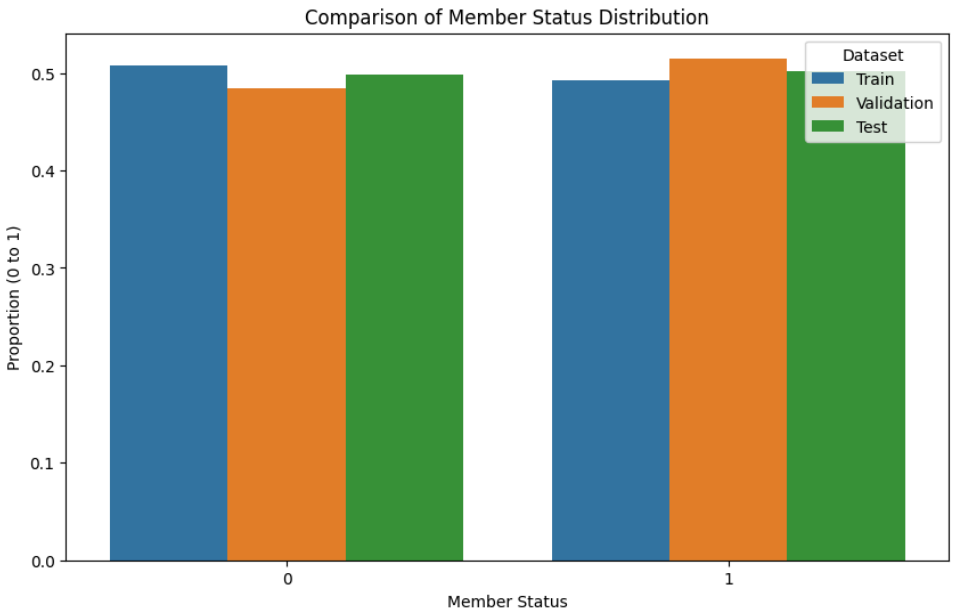

Figure 8: *Comparison of Household Member Status Distribution*

Second, membership status is evenly distributed across the three splits, preventing bias in pricing or promotion-related behaviour. Similarly, price sensitivity segments and life stage groups are proportionally represented across training, validation, and test sets, ensuring that demographic and behavioural diversity is preserved throughout.

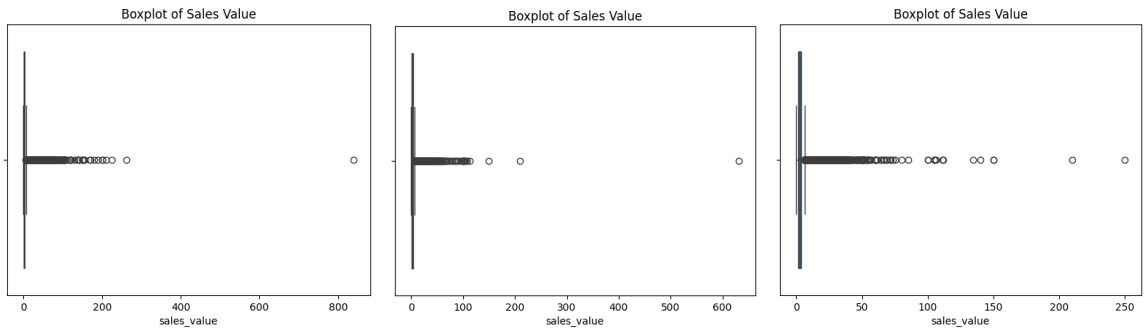

Figure 9: *Comparison of Sales Value Distribution Across Temporal Data Splits (Train, Validation, Test)*

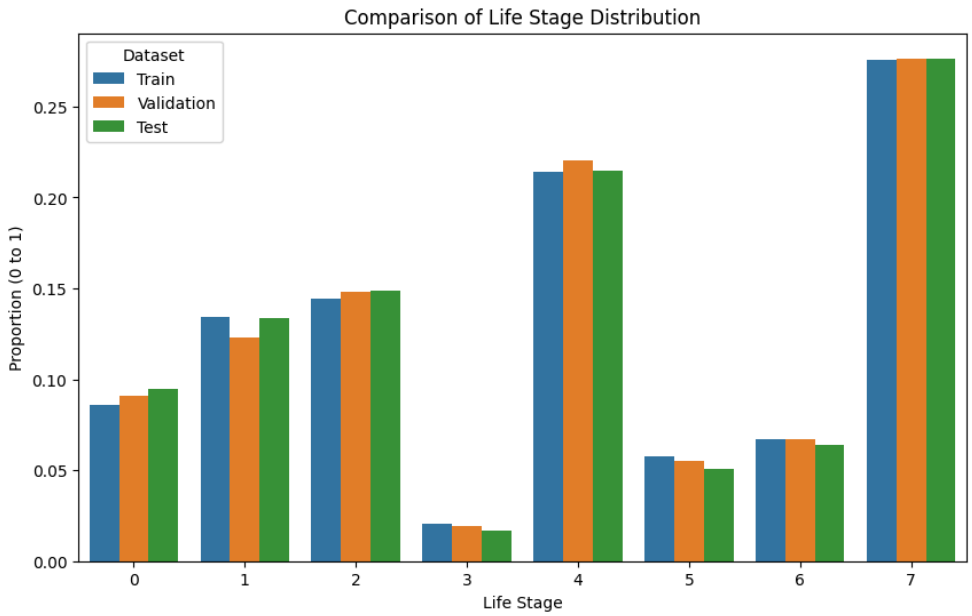

Figure 10: *Comparison of Household Life Stage Distribution*

Finally, boxplots of sales value exhibit similar distributions across all splits, indicating consistent transaction value patterns with no major shift in purchase behavior.

Overall, these checks confirm that the temporal split maintains both chronological integrity and distributional consistency, enabling a realistic yet stable evaluation of model performance.

Since all 728 households appear in both the training and validation sets, there are no truly cold users (i.e., users with no prior interaction history). As a result, the cold-user pathway in the Multi-Branch Two-Tower architecture would not naturally receive any gradient signal during training and would remain undertrained.

To address this, we simulate cold-user conditions by randomly designating 20% of users in each split as cold (is_warm = 0). This flag is passed through the dataset and used during the forward pass to route users to either the warm or cold user tower.

This controlled simulation ensures that both pathways are actively trained, allowing the model to learn meaningful representations for cold users. Without this mechanism, the cold-user tower would receive little to no updates and risk collapsing into a degenerate solution.

2. Models and Performance

2.1 Problem Formulation

The project formulates recommendations as a binary classification task over household-product pairs using implicit feedback. Observed purchases are treated as positive labels, while negatives are sampled from unobserved items (4:1 ratio). Each model outputs a purchase probability for a given pair.

To evaluate basket-level recommendations, we adopt a 1-vs-99 protocol (He et al., 2017): for each held-out positive interaction, 99 negatives are sampled from the training pool and ranked together. The top-K items form the recommended basket, and performance is measured by whether these items are actually purchased. Negative samples are fixed across models for fair comparison.

2.2 Model Architectures

We implemented six models to evaluate different sources of information:

- MF and NCF: interaction-based baselines
- User-Feature Model: replaces user embeddings with engineered features
- Item-Feature Model: replaces item embeddings with product features
- Two-Tower Model: fully feature-based user and item encoders
- Multi-Branch Two-Tower: separate pathways for warm (ID + features) and cold items (features only), with 15% cold-negative sampling

2.3 Training Protocol

All models are trained using BCEWithLogitsLoss with AdamW and early stopping on validation AUC (patience = 3, max = 20 epochs). The best checkpoint is restored for test evaluation.

2.4 Test Results

Model	AUC	Recall@10	MRR@10
MF	0.8682	0.6252	0.3313
NCF	0.8865	0.6821	0.3827
User-Feature	0.9305	0.7299	0.3925

Item-Feature	0.8765	0.6371	0.3376
Two-Tower	0.8492	0.6242	0.3261
Multi-Branch Two-Tower	0.8484	0.6251	0.3223

Table 2: *Test set performance across all models (K = 10, 1-vs-99 evaluation protocol)*

Key Finding 1: The User-Feature Model dominates, outperforming all baselines across AUC, Recall@10, and MRR@10. This shows that engineered demographic and behavioral features provide stronger user representations than interaction-based embeddings alone.

Key Finding 2: Item features provide limited gains, while fully feature-based models (Two-Tower) underperform. This suggests that collaborative signals from item IDs remain critical for recommending warm items.

2.5 Additional Analyses

2.5.1 Cold Start Analysis

Cold items (< 50 interactions) show extremely low recall@10 (0.0021) across all models, indicating failure to surface them in top K recommendations. However, the User-Feature Model achieves an AUC of 0.9518, meaning it can distinguish relevant cold items but fails to rank them highly. This highlights a key issue: cold items lose out to popular items due to lower absolute scores. The difficulty in ranking cold items reflects a well-known limitation of latent collaborative filtering models, which rely heavily on historical interaction data and degrade in sparse settings (Volkovs et al., 2017).

2.5.2 Repeat vs New Items Analysis

Most models recommend ~50-55% repeat items. In contrast, the Two-Tower Model is highly exploratory, recommending only 23.6% repeat items (76.4% new items). This is due to the absence of item ID embeddings. While beneficial for discovery, its lower AUC indicates weaker relevance.

2.5.3 Demographic Segment Analysis

The User-Feature Model demonstrates the most consistent performance across demographic segments, with minimal AUC variation across income, life stage, and price sensitivity groups.

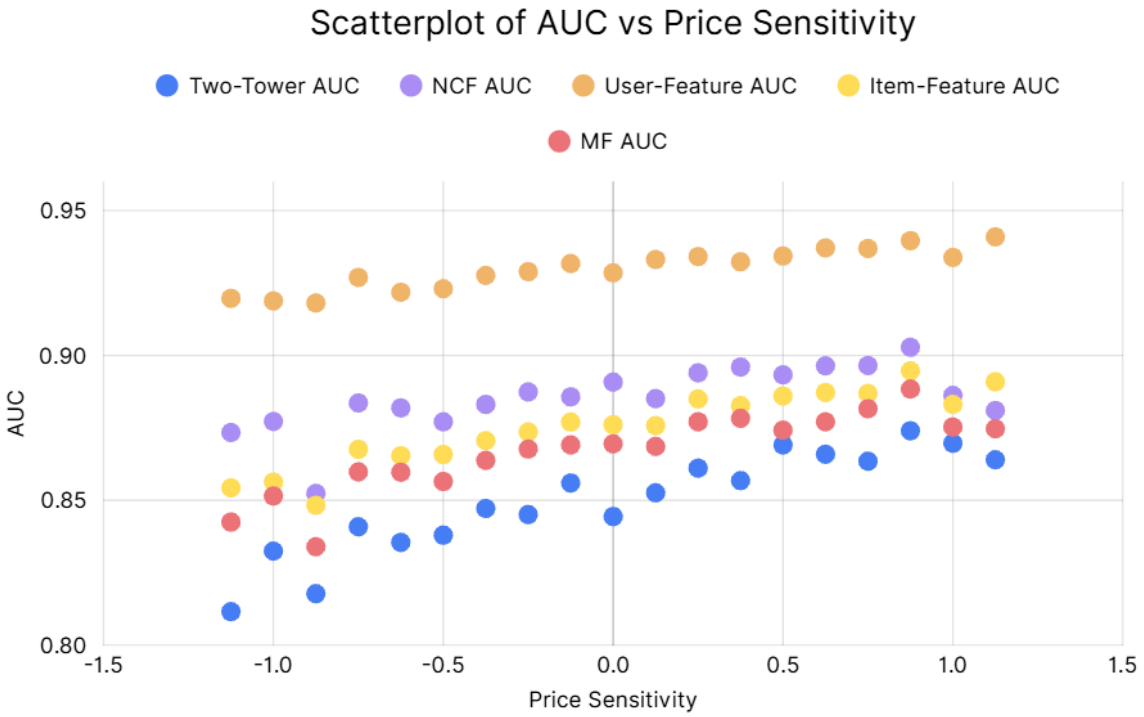

Figure 11: *Scatterplot of AUC vs Price Sensitivity*

Across price sensitivity, all models exhibit a clear upward trend in AUC, indicating that price-sensitive users display more predictable purchasing behaviour. This suggests that price-related features play a significant role in driving model performance.

2.5.4 Calibration by Segment

The User-Feature Model achieves near-perfect calibration (ECE ~ 0.01), while all other models show high error (~0.40 - 0.45). This indicates that its predicted probabilities closely reflect true purchase likelihood.

2.5.5 Two-Tower Model Ablation Study

Removing item features leads to a 70.7% drop in recall@10, confirming that item-side features are the primary driver of performance. Removing user features slightly improves performance, suggesting noise in user feature representation.

2.6 Limitations

2.6.1 Limited Modelling of Temporal Dynamics

The current models do not explicitly account for temporal patterns in customer purchasing behaviour, such as seasonality or day-of-week effects. In a retail setting, user-item

interactions are often time-dependent, and ignoring this may reduce the model's ability to capture when a product is likely to be purchased (Koren, 2010). As a result, recommendations may be less responsive to shifting consumer needs and short-term behavioural trends. A possible improvement would be to include temporal features such as recency or promotion periods, or to adopt time-aware recommendation models (Koren, 2010) that can better capture changes in user behaviour over time.

2.6.2 Popularity Bias

The current models exhibit a tendency towards popularity bias, where frequently purchased items are disproportionately recommended (Abdollahpouri, 2019). This limits the visibility of long-tail products and contributes to the observed difficulty in surfacing cold items in top-K recommendations. A potential solution is to incorporate popularity debiasing techniques, such as re-weighting loss functions or applying post-ranking adjustments that penalize overly popular items. Additionally, re-ranking strategies that explicitly balance relevance and diversity, such as those proposed by Abdollahpouri (2019), could improve exposure for long-tail items by personalizing the degree of debiasing applied to each user.

3. Contribution and Justification

3.1 Self-Evaluation Table

Contribution Aspect	Level	Effort	Effectiveness	Summary
1. Valuable datasets	Basic	Moderate	Level 3	Integrated transaction, product, and demographic data across 801 households, 92400 products, and 2 years of purchase history
2. Feature engineering	Optional	High	Level 3	OpenAI text embeddings + PCA, Shannon entropy, time-decay frequency, co-occurrence, life stage interaction, price sensitivity composite index
3. ML methods/architecture	Optional	High	Level 3	Multi-Branch Two-Tower with item-coldness routing; ablation study; 1-vs-99 evaluation protocol
4. ML output understanding	Optional	High	Level 3	Cold start analysis, repeat vs new item analysis, demographic segment analysis, calibration analysis, ablation study
5. Smart use of AI tools	—	High	Level 3	Used AI tools for code review, pipeline architecture design confirmation, and debugging

Table 3: *Self-Evaluation Table*

3.2 Aspect 1: Dataset Quality and Integration

This project uses the Dunnhumby The Complete Journey dataset (Gonen, 2014), a publicly available retail panel dataset containing genuine two-year purchase histories for over 2500 real households. Three structurally distinct source tables were joined to create a unified learning representation: transaction_data provides the interaction signal, product provides item-level taxonomy and brand information, and hh_demographic provides household-level socioeconomic attributes. The multi-table integration produces a substantially richer learning signal than any single table could provide in isolation, because demographic features alone cannot predict which items will be bought, transaction patterns alone cannot generalise to unseen items, and product metadata alone carries no user-specific signal.

Although no new data was crawled or collected from external sources, the data preparation stage was central to the project's technical complexity. Careful analysis of NA alignment patterns across demographic columns confirmed that 116,842,900 rows with missing values arose from a coherent block of households, allowing for them to be removed without hurting

the explanatory power of the dataset. The identification and removal of systematic data quality issues, including zero-price promotional transactions, refund-like negative transactions, and above-shelf-price anomalies, required domain reasoning about what constitutes a genuine purchase signal. The resulting cleaned dataset of 1.28 million rows across 92339 products and 728 households provides sufficient density for NCF models to learn meaningful latent representations.

3.3 Aspect 2: Feature Engineering

Our feature engineering goes substantially beyond descriptive statistics to incorporate domain knowledge, representation learning, and composite signal construction.

3.3.1 Time-Decay Frequency

Rather than counting raw purchase frequency, we weigh each transaction by an exponential decay factor (Ding & Li, 2005), defined as exp(-lambda(T_max = t)) with lambda = 0.05, assigning a higher weight to recent transactions. This encodes the domain insights that recent grocery preferences are more predictive than historical ones. With $\lambda = 0.05$, the implied half-life is approximately 14 weeks, meaning that the contribution of a transaction falls by half after that period. This allows the feature to capture gradual preference drift over time and improves on simple frequency counts used in baseline recommender systems by incorporating recency directly into the user representation.

3.3.2 Life Stage Interaction Feature

Rather than treating age bracket, household size, and child presence as independent inputs, we constructed a composite life_stage feature to explicitly encode their interaction. This feature represents household behavioural archetypes, such as Young Family, Older Family, Young Single, Middle-aged Single, Pre-retirement Single, Retired/Older Single, and Couple/Other, reflecting domain knowledge that purchasing behaviour is shaped by life-stage rather than any single demographic attribute alone. This reduces feature dimensionality while preserving meaningful cross-variable structure. In addition, life_stage was retained over household_size_desc due to their relatively high collinearity of 0.751, with life_stage offering greater behavioural interpretability.

3.3.3 Shannon Entropy for Product Diversity

We use Shannon Entropy (Shannon, 1948) to quantify the variety and predictability of each household's purchasing behaviour. By calculating the entropy of commodity descriptions at the household level, we capture a shopping diversity score that distinguishes habitual

consumers from broad-spectrum explorers. This feature provides a critical behavioural signal where low-entropy users receive high-precision recommendations based on recurring needs, while high-entropy users are served a more diverse set of items. For cold start users, the application of a global entropy ensures a stable behavioural baseline, allowing the model to generalise based on aggregate population trends until individual purchase signatures appear.

3.3.4 Item Co-Occurrence Network Features

We construct three item graph statistics from the training basket data: log_co_purchase (how often the item co-appears with other items), avg_basket_size (the typical trip size when this item is bought), and basket_centrality (fraction of all trips containing this item). Together, these features capture item complementarity and trip-type association in a way that simple item-level frequency counts cannot.

3.3.5 OpenAI Text Embeddings with PCA Compression

Product descriptions are formed as ‘commodity_desc: sub_commodity_desc’ and embedded using OpenAI’s text embeddings-3-large (1536 dimensions). PCA (256 components, fit on train item embeddings only) compresses these to a manageable size while retaining the majority of semantic variance. This representation learning approach captures fine-grained product similarity that categorical encodings cannot express and enables the Item-Feature and Two-Tower Models to generalise across semantically related items, even for cold items with no interaction history.

3.3.6 Price Sensitivity Composite Index

Rather than using raw discount rates or coupon usage as separate features, we construct a price_sensitivity_index by quantile-ranking user_coupon_rate and user_avg_discount independently (10 quantile bins each, fit on train) and then averaging the two ranks. This creates a stable ordinal composite on a 0 - 9 scale that is robust to outliers and comparable across users with different purchase volumes.

3.4 Aspect 3: ML Architecture and Methods

3.4.1 Progressive Model Family for Systematic Comparison

We design a systematic family of six model architectures that isolate the contribution of different information sources: MF and NCF are pure collaborative filtering baselines, User-Feature and Item-Feature Models augment one side with content features, the

Two-Tower Model is a pure content-based encoder, and the Multi-Branch Two-Tower Model adds cold-item routing. This structured design makes it possible to directly attribute performance differences to specific architectural choices, rather than relying solely on aggregate metrics across unrelated models.

3.4.2 Multi-Branch Two-Tower with Item-Cold Routing

This is our most significant architectural innovation. Unlike the standard Two-Tower Model, which uses a single item pathway for all items regardless of training history, the Multi-Branch Two-Tower explicitly routes items through two specialized pathways based on interaction density. The warm item tower has access to both a learned ID embedding and item features, allowing it to exploit memorised purchase patterns for frequently seen items. The cold item tower receives only item features, with no embedding lookup, forcing it to rely exclusively on content-based generalisation for items that have accumulated fewer than 50 interactions. This design is architecturally distinct from post-hoc slicing of a standard Two-Tower Model on warm and cold test subsets because the two towers are explicitly trained to solve different sub-problems and can learn different optimal weight configurations for each.

3.4.3 Cold-Item Negative Injection

Standard uniform negative sampling draws exclusively from the training item pool, meaning cold items (those with < 50 interactions) are sampled as negatives with a probability proportional to their training set frequency. Because cold items are rarely purchased, they appear as negatives very infrequently under uniform sampling. The cold tower of Multi-Branch Two-Tower then receives nearly no gradient signal, telling it to score cold items low. By reserving 15% of negatives for cold items, we ensure the cold item tower receives sufficient negative gradient signal to learn to assign low scores to cold items that are irrelevant to the user. This design choice is motivated by the sampling-bias-corrected approach proposed by (Yi et al., 2019), adapted here to the cold-start context.

3.4.4 1-vs-99 Evaluation Protocol with Fixed Candidate Sets

We implement the 1-vs-99 evaluation protocol standard in recommender systems research: for each held-out positive interaction, 99 negatives are sampled from the train item pool, and all 100 candidates are ranked. This ensures that differences in metric values between models reflect true ranking quality differences, not differences in the specific negatives drawn. Computing metrics over candidate sets of 100 is also more realistic than scoring against the full item catalogue, as the 92339-item catalogue is too large for meaningful top 10 recall comparisons.

3.5 Aspect 4: Understanding ML Output and Business Implications

3.5.1 Cold Start Analysis

All six models are evaluated on the subset of test interactions where the ground truth positive item has fewer than 50 training interactions. This directly quantifies each model's ability to support newly introduced products, which is a commercially critical requirement for any grocery recommendation system because retailers regularly launch new stock-keeping units.

3.5.2 Repeat vs New Item Analysis

For each model, we compute the per-user fraction of the top 10 recommendations that consist of items the household has already purchased during training. A model with a high repeat ratio is conservative and personalised to history, while a model with a low repeat ratio is more exploratory but may sacrifice precision. For a grocery retailer, this trade-off has direct business implications: repeat recommendations drive basket efficiency (customers buy what they need), while novel recommendations drive basket expansion (customers discover new products). This analysis gives the retailer an empirical basis for deciding which model to deploy, depending on whether the current business priority is retention of established purchasing habits or expansion into new categories.

3.5.3 Demographic Segment Analysis

Model performance is reported separately for each of the income brackets, life stage categories, price sensitivity details, and average spending tiers present in the test set. Segmentation by these user-level attributes tests whether models that incorporate demographic features produce more equitable recommendations across the household population, or whether ID-based models achieve comparable performance across segments simply because the training data contains sufficient interaction density for all groups.

3.5.4 Calibration by Segment (ECE)

Calibration measures the alignment between a model's predicted purchase probabilities and the actual empirical frequencies of purchase. Expected calibration error is defined as the expected absolute difference between the mean predicted probability in a bin and the fraction of positive labels in that bin. Calibration curves are plotted separately for each model across income, life stage, and price sensitivity segments. A well-calibrated model is practically important for a retailer that uses predicted probabilities to make downstream decisions, such as setting minimum probability thresholds for including an item in a

recommendation panel or allocating marketing budget to specific product categories. A model that recommends items accurately but produces systematically overconfident or underconfident probabilities would lead to poor threshold-based decisions even if its metrics are high.

3.5.5 Two-Tower Ablation

The ablation study isolates the marginal contribution of each input component of the Two-Tower Model by systematically replacing individual inputs with zero vectors and measuring the resulting change in recall@10. The conditions tested are the full model, the model with user features zeroed, the model with item features zeroed, the model with both user inputs zeroed (equivalent to removing the user tower), and the model with both item inputs zeroed (equivalent to removing the item tower). Comparing the performance drops across these conditions provides interpretable, quantified evidence for architectural decisions and directly justifies the computational cost of calling the OpenAI embedding API, since the magnitude of the drop when item features are zeroed shows exactly how much the semantic embeddings contribute to overall recommendation quality.

3.6 Aspect 5: Smart Use of AI Tools

AI tools (specifically Claude and Gemini) were used throughout this project as a competency multiplier rather than a replacement for understanding. Specific uses include:

- Pipeline architecture review: AI tools were used to review the recommendation pipeline for data leakage issues (e.g., whether negative sampling used the full item pool vs train-only pool, whether scalers were fit on train only, whether val/test deduplication was needed before the 1-vs-99 protocol)
- Architecture design: discussions about the difference between evaluating one model on warm/cold slices vs training a model with explicit warm/cold routing pathways informed the Multi-Branch Two-Tower design

In all cases, the team verifies the generated or reviewed code against the project's theoretical requirements, and every team member understands the purpose of each pipeline component.

4. References

Abdollahpouri, H. (2019). Popularity Bias in Ranking and Recommendation. *Proceedings of the 2019 AAAI/ACM Conference on AI, Ethics, and Society*, 529–530. https://doi.org/10.1145/3306618.3314309

Catalina (2026). Machine learning use cases in retail: Machine learning in retail. Catalina. (2026). https://www.catalina.com/perspectives-blog/machine-learning-retail

Ding, Y., & Li, X. (2005). Time Weight Collaborative Filtering. *Proceedings of the 14th ACM International Conference on Information and Knowledge Management*, 485–492. https://doi.org/10.1145/1099554.1099689

Gonen, F. (2014). *Dunnhumby The Complete Journey* [Data set]. dunnhumby. https://www.kaggle.com/datasets/frtgnn/dunnhumby-the-complete-journey/

He, X., Liao, L., Zhang, H., Nie, L., Hu, X., & Chua, T.-S. (2017). Neural Collaborative Filtering. Proceedings of the 26th International Conference on World Wide Web, 173–182. https://doi.org/10.1145/3038912.3052569

Koren, Y. (2010). Collaborative filtering with temporal dynamics. *Communications of the ACM*, *53*(4), 89–97. https://doi.org/10.1145/1721654.1721677

Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, *27*(3), 379–423. https://doi.org/10.1002/j.1538-7305.1948.tb01338.x

Volkovs, M., Yu, G. W., & Poutanen, T. (2017). Content-based Neighbor Models for Cold Start in Recommender Systems. *Proceedings of the Recommender Systems Challenge 2017*, 1–6. https://doi.org/10.1145/3124791.3124792

Yi, X., Yang, J., Hong, L., Cheng, D. Z., Heldt, L., Kumthekar, A., Xu, Z., Li, C., & Chi, E. (2019). *Sampling bias-corrected neural modeling for large corpus item recommendations*. Proceedings of RecSys 2019, pp. 269-277, https://doi.org/10.1145/3298689.3346996